

Design Note: News Recommendation System (Assignment 1)

1. What was built and key design choices

We built a **Two-Stage (L1/L2) Hybrid Neural Recommendation System** optimized for both the MIND (English) and EB-NeRD (Danish) datasets.

Key Design Choices:

- **Polars Data Pipeline:** Instead of Pandas, the entire data pipeline was built in Polars. This allowed us to process the massive 15M+ impression logs in seconds rather than minutes, completely eliminating the Out-Of-Memory (OOM) crashes that typically plague recommendation data processing.
- **L2 Ranking vs L1 Retrieval:** Rather than performing a slow, global retrieval across all 120,000 articles, we recognized that the Codabench test sets already provide pre-filtered L1 candidates (the ~50 articles in the `impressions` column). Our architecture focuses entirely on the **L2 Reranking** step, scoring only the provided candidates.
- **GPU Batch Matrix Multiplication:** To compute Semantic Similarities, doing a for-loop across 6 million users takes >9 hours. We pre-loaded the 120,000 dense article embeddings (paraphrase-multilingual-MiniLM-L12-v2) directly into the T4 GPU's VRAM. By chunking 1,500 users at a time and using `torch.mm()`, we accelerated candidate scoring to under 2 minutes.
- **LightGBM LambdaMART:** We used LightGBM to fuse 6 handcrafted features (Semantic Score, Lexical Overlap, Log Popularity, History Length, Category Match, and Inverse Position) into a final ranking score optimized for nDCG.

2. Alternatives considered and why you chose what you did

- **Global BM25 vs Lexical Overlap:** We originally attempted to build a global BM25 index over all 120,000 articles for Lexical candidate generation.
- *Why we rejected it:* Constructing per-impression BM25 queries for 6 million users took hours on Kaggle CPUs. More importantly, global BM25 retrieved articles that had near-zero overlap with the 50 candidates Codabench expects, resulting in a random baseline score (AUC 0.51).
- *What we chose:* We replaced global BM25 with an in-memory **Lexical Overlap** feature (Jaccard intersection between user history tokens and candidate tokens). This provided the exact same "keyword match" signal but computed locally in microseconds.
- **Fine-Tuning Encoders vs Pre-Trained:** We considered fine-tuning models like NRMS or NAML.
- *Why we rejected it:* Fine-tuning requires massive dataset processing (50GB+) and significant GPU training time.

- *What we chose:* We used a zero-shot, pre-trained multilingual encoder (paraphrase-multilingual-MiniLM-L12-v2). This gave us competitive semantic representations out-of-the-box for both English and Danish, allowing us to invest our compute time into the LightGBM reranker instead.

3. Observations from experiments (lexical vs. semantic, dataset differences)

We ran our local evaluation harness (`src/evaluation/evaluate.py`) on the validation splits and observed the following:

- **Lexical vs Semantic:** Relying purely on semantic scores led to "topic drift" (e.g., recommending basketball when the user wanted soccer). Conversely, pure Lexical Overlap failed to catch synonyms. When fed into LightGBM, the model learned to heavily weight the **Semantic Score** (Feature Importance: ~501,000) for general ranking, but used **Lexical Overlap** (Feature Importance: ~48,000) as a highly precise tie-breaker for identical topics.
- **MIND (English) Metrics:**
 - nDCG@5: 0.3163 | nDCG@10: 0.3725 | AUC: 0.6541 | MRR: 0.3412
- **EB-NeRD (Danish) Metrics:**
 - nDCG@5: 0.3452 | nDCG@10: 0.4011 | AUC: 0.6720 | MRR: 0.3645
- **Dataset Differences:** The EB-NeRD dataset consistently achieved higher nDCG metrics. This is likely because Ekstra Bladet (EB-NeRD) articles are more hyper-localized to Danish news, meaning popular articles have a much higher base click-through rate, making "Log Popularity" a stronger predictive signal compared to the highly diverse, global MIND dataset.
- **Beyond-Accuracy:** The L2 approach maintained a healthy **Novelty (0.68)**, proving that the model wasn't simply recommending the most popular items to every user.

4. Where your pipeline breaks at 10x scale

Our current L2 architecture relies heavily on loading all 120,000 dense semantic embeddings directly into the GPU's VRAM for lightning-fast batch matrix multiplication. This currently occupies about ~2GB of the 14GB available on a Kaggle T4 GPU.

The Breakage: If the article catalog scaled 10x (to 1.2 million articles), the dense embedding matrix (1.2M x 384 x 4 bytes) would consume over 18GB of VRAM, triggering a strict CUDA OutOfMemory crash on standard free-tier hardware.

The Solution at 10x Scale: To fix this, we would have to abandon full dense matrix multiplication. We would transition to an Approximate Nearest Neighbor (ANN) index like FAISS IndexIVFFlat, storing the bulk of the embeddings in standard CPU RAM (or disk) and passing only quantised, highly probable clusters to the GPU for final distance calculation.

(AI Usage Logs and Leaderboard Screenshots are included in the @screenshots directory of the submission).